32

BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

CopyOnWriteArrayList: Anlık Görüntü (Snapshot) İterasyonu & Yazma Cezası

CopyOnWriteArrayList: Snapshot Iteration, Write Penalties & When to Use It

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              COPY-ON-WRITE (COW) DİZİ KLONLAMA MEKANİZMASI │
└──────────────────────────────────────────────────────────┘

MEVCUT DİZİ (İç Array): [ "A", "B", "C" ] ◄── Binlerce Okuyucu Thread
                                              (SIFIR KİLİT! ULTRA HIZLI!)

BİR YAZICI THREAD GELDİ (add("D")):
1. ReentrantLock AL
2. Dizinin YEPYENİ BİR KOPYASINI oluştur: new Object[eski.length + 1]
3. Elemanları kopyala ve yeni elemanı ekle: [ "A", "B", "C", "D" ]
4. İçsel volatile referansı yeni diziye bağla: array = newArray;
5. ReentrantLock BIRAK

SONUÇ: Okuyucular gezinirken (Iterator) ASLA ConcurrentModificationException ALMAZ!
ANCAK sık yazma yapılırsa devasa bellek kopyalama maliyeti oluşur!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirketin telefon rehberini düşünün. 1000 çalışan gün boyunca rehbere bakıp numaraları arar (Okuyucular). Kimsenin birbirini beklemesine gerek yoktur. Ama şirkete yeni bir personel girdiğinde, sekreter mevcut rehberin üzerine karalama yapmaz! Masasındaki fotokopi makinesinden rehberin koca bir kopyasını çıkarır, yeni personeli o yeni kağıda yazar ve panodaki eski kağıdı yenisiyle değiştirir! Eski kağıdı okumakta olanlar rahatsız olmaz. Ama her saniye yeni personel giriyorsa, fotokopi makinesi yanar!

🇬🇧 ENGLISH (ELI5)

Think of a published corporate phone directory. Hundreds of employees browse it simultaneously without blocking (Readers). When an employee joins the company, the receptionist does not scribble on the active sheet. Instead, the receptionist photocopies the entire directory, appends the new name to the duplicate, and magnetically snaps the new copy onto the board! Readers actively holding the old sheet finish unperturbed. But if you have 10,000 updates a second, the photocopier catches fire!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CopyOnWrite Mimarisi ve Kullanım Alanları: 1) Snapshot İteratör Garantisi: `iterator()` çağrıldığı anda o anki dizinin referansını alır. Başka bir thread listeye 1 milyon eleman eklese veya silse bile, iteratör kendi anlık görüntüsünü (Snapshot) taramaya devam eder; asla `ConcurrentModificationException` fırlatmaz. 2) Ağır Yazma Cezası (O(N) Allocation): Her `add()`, `set()`, `remove()` işlemi dizinin tamamını `Arrays.copyOf` ile yeniden tahsis eder. 3) Ne Zaman Kullanılmalıdır?: 'Okuma Çok (%99+), Yazma Çok Nadir (%1)' olan sistemlerde: Event Listener listeleri, filtre zincirleri ve güvenlik kuralı tabloları için mükemmeldir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CopyOnWrite Architectural Economics: 1) Fail-Safe Snapshot Iteration: The iterator holds a reference to the exact array state at the moment `iterator()` was invoked. Mutations occurring during iteration swap the internal `volatile Object[] array` reference without modifying the iterator's snapshot array, eliminating `ConcurrentModificationException` and the need for locks during reads. 2) O(N) Write Amplification: Every write operation acquires an exclusive `ReentrantLock` and clones the underlying backing array (`Arrays.copyOf()`), inducing substantial GC allocation pressure. 3) Domain Fit: Exclusively viable for read-dominant collections such as observer listener lists, routing tables, and security filter chains.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Event Listener Havuzunda CopyOnWriteArrayList Kullanımı
import java.util.concurrent.CopyOnWriteArrayList;

public class SecurityAuditNotifier {
    // Listener listesi: Okuma çok sık, yeni listener ekleme çok nadirdir
    private final CopyOnWriteArrayList<String> securitySubscribers = new CopyOnWriteArrayList<>();

    public void subscribe(String serviceName) {
        securitySubscribers.add(serviceName); // Dizi kopyalanarak yazılır
    }

    public void broadcastSecurityAlert(String alertMessage) {
        // SIFIR KİLİT! İterasyon anında liste değişse bile patlamaz!
        for (String subscriber : securitySubscribers) {
            IO.println("Alert yollandı -> " + subscriber + ": " + alertMessage);
        }
    }

    void main() {
        var notifier = new SecurityAuditNotifier();
        notifier.subscribe("FraudService");
        notifier.subscribe("AuthGateway");
        notifier.broadcastSecurityAlert("CVE-2026-9999 tespit edildi!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı '100.000 elemanlı bir listeye sürekli veri ekleyeceğimiz bir sistemde CopyOnWriteArrayList kullanabilir miyiz?' diye sorar. Yanıt: 'KESİNLİKLE HAYIR! Her eleman eklemede 100.000 elemanlık dizi baştan kopyalanır; bellek ve işlemci tükenir, sistem OOM veya Garbage Collection duraklamalarıyla felç olur!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Performance Trap: 'Can we use CopyOnWriteArrayList for high-throughput stream ingestion?' Answer: 'ABSOLUTELY NOT! Appending 100,000 items creates 100,000 full array allocations and copies, consuming gigabytes of transient heap and collapsing GC pause times. Use ConcurrentLinkedQueue or Collections.synchronizedList instead.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CopyOnWriteArrayList okuma işlemlerinde sıfır kilit maliyetiyle çalışır.
- İteratörler anlık görüntü (snapshot) üzerinden gezinir; CME hatası imkansızdır.
- Yazma işlemleri diziyi baştan kopyaladığı için sık yazılan sistemlerde kullanılmaz.

🎯 Key Takeaways for the Interview

- CopyOnWriteArrayList provides lock-free, volatile-read performance for all queries.
- Iterators traverse an isolated array snapshot, guaranteeing immunity to CME.
- Writes incur full array re-allocation; strictly avoid in write-heavy environments.